Chuleta CSS: flex, grid y todo lo de Festival Marea

Oct 29, 2026 · @Alin

1. Cómo decidir qué usar

Antes de escribir CSS, hazte siempre tres preguntas, en este orden. Casi todos tus errores venían de saltarte la primera.

1. **¿Quién es el padre?** Busca en el HTML el elemento que contiene *directamente* lo que quieres colocar. `display: flex` o `display: grid` va **ahí**, nunca en los hijos.
2. **¿Flex o grid?** Una fila o una columna de cosas que se alinean → flex. Columnas que tienen que cuadrar entre sí → grid.
3. **¿Qué necesita cada hijo?** Con el padre hecho, mira si algún hijo necesita algo propio (`flex: 1`, `margin-top: auto`, `order`, `align-self`).

Sección de Marea	Padre grid	Padre flex
Cabecera	—	`.cabecera__interior`, `.menu__lista`
Hero	—	`.hero__interior`, `.hero__acciones`
Datos clave	`.datos__lista`	`.dato`
Cartel	`.cartel__rejilla`	`.tarjeta`, `.tarjeta__cuerpo`
Escenarios	`.bloque`	`.bloque__texto`
Entradas	`.planes`	`.plan`, `.plan__lista`
Pie	`.pie__interior`	`.pie__lista`, `.pie__legal-interior`

Regla rápida: **grid para el reparto general, flex para lo de dentro de cada pieza**. Se pueden anidar sin límite.

2. Flexbox: una dimensión

Flex coloca los hijos **en una línea** (fila o columna) y deja que el contenido decida cuánto ocupa cada uno. Se activa en el padre con `display: flex`; los hijos pasan a ser "elementos flex".

Idea clave: los dos ejes. El *eje principal* es la dirección (horizontal en `row`, vertical en `column`). El *eje cruzado* es el perpendicular. Si cambias a `column`, los ejes se intercambian.

Propiedades del padre

Propiedad	Qué hace	Ejemplo en Marea
`display: flex`	Activa flex: los hijos se ponen en fila	Cabecera, hero, tarjeta
`flex-direction`	`row` (fila, por defecto) o `column` (columna)	Hero y tarjeta en `column`
`justify-content`	Reparte en el **eje principal**: `flex-start`, `center`, `space-between`, `flex-end`	Cabecera con `space-between`; barra legal del pie
`align-items`	Alinea en el **eje cruzado**: `stretch` (por defecto), `center`, `flex-start`, `flex-end`	Cabecera con `center`; hero con `center`
`gap`	Hueco fijo entre hijos	Menú, botones del hero
`flex-wrap`	`nowrap` (por defecto) o `wrap` para que salten de línea	No lo usamos (solo escritorio)

Propiedades del hijo

Propiedad	Qué hace	Ejemplo en Marea
`flex: 1`	El hijo ocupa **todo el espacio sobrante**	`.menu`, `.tarjeta__cuerpo`, `.plan__lista`
`align-self`	Sobrescribe el `align-items` del padre solo para ese hijo	Etiqueta "Más vendida" con `flex-start`
`margin-top: auto`	En una columna flex, empuja el elemento al fondo	Botón de la tarjeta
`margin-left: auto`	En una fila flex, empuja el elemento a la derecha	Primer y último `<li>` del menú
`order`	Cambia la posición visual sin tocar el HTML	Ver sección Grid

Cómo funciona `margin: auto` en flex

En flex, un margen `auto` se lleva **todo el espacio libre**. Si hay dos márgenes `auto` (el primer y el último `<li>` del menú), se reparten el espacio por igual: los enlaces quedan centrados y el botón en el extremo derecho.

Cuándo usar flex

- Una barra de navegación (logo | menú | botón).
- Centrar algo en vertical y horizontal (hero).
- Apilar elementos en columna y empujar uno al fondo (tarjetas, planes).
- Dos textos, uno a cada lado (barra legal).
- Botones en fila con separación igual.

3. Grid: dos dimensiones

Grid define **columnas (y filas)** y coloca los hijos en esa rejilla. También se activa en el padre. Los hijos se van colocando por orden, sin que tengas que decir dónde va cada uno. Las filas se crean solas y **los elementos de una misma fila se igualan en altura**.

Propiedades

Propiedad	Qué hace	Ejemplo en Marea
`display: grid`	Activa grid en el padre	Datos, cartel, escenarios, planes, pie
`grid-template-columns`	Define las columnas	`repeat(3, 1fr)`, `1fr 1fr`, `2fr 1fr 1fr 1fr`
`gap`	Hueco entre filas y columnas	Cartel y planes con `--espacio-4`
`align-items`	Alinea los hijos en vertical dentro de su celda	Bloques de escenarios con `center`
`place-items`	Atajo: `align-items` + `justify-items`	Alternativa para centrar el hero
`order` (en el hijo)	Cambia el orden visual	`.bloque--inverso .bloque__visual` con `-1`

La unidad `fr` y `repeat()`

- `1fr` = "una parte del espacio disponible". Dos columnas `1fr 1fr` son iguales; `2fr 1fr` la primera es el doble de ancha.
- `repeat(3, 1fr)` es un atajo de `1fr 1fr 1fr`.
- En el pie, `2fr 1fr 1fr 1fr` hace que la columna de marca mida el doble que las demás.

Cómo funciona `order`

Por defecto todos los hijos valen `order: 0` y se colocan en el orden del HTML. Con `order: -1` un hijo pasa a ir el primero. Así el bloque "Escenario Pinar" pone la imagen a la izquierda **sin tocar el HTML**.

Qué pasa si falta o sobra una tarjeta

Con 7 tarjetas en una rejilla de 3 columnas, la última fila tiene una sola tarjeta y **no se estira**: ocupa solo su columna.

Cuándo usar grid

- Tarjetas o planes en filas de la misma anchura.
- Un bloque de dos columnas (texto | imagen).
- Un pie con columnas de distinto ancho.
- Cualquier cosa donde las columnas deban cuadrar entre filas.

Flex vs grid, en una frase

Flex deja que **el contenido** decida el reparto. Grid deja que **tú** definas la rejilla y el contenido se adapta.

4. Texto y tipografía

Estas propiedades afectan al **texto**, no a las cajas. Por eso `text-align: center` no sirve para centrar una caja, y `justify-content` no centra el texto de un párrafo.

Propiedad	Qué hace	Valores habituales	Ejemplo en Marea
`text-align`	Alinea las **líneas de texto** dentro de su caja	`left`, `center`, `right`	Hero y datos con `center`
`font-size`	Tamaño de letra	`var(--texto-1)`, `clamp()`	Título del hero con `--texto-3xl`

Propiedad	Qué hace	Valores habituales	Ejemplo en Marea
`font-weight`	Grosor	`400` normal, `700` fuerte, `800` negro	Títulos con `--peso-fuerte` o `--peso-negro`
`font-family`	Tipo de letra	`var(--fuente-titulo)`	Precios y valores de datos
`color`	Color del **texto** (y se hereda a los hijos)	`var(--color-tinta)`	`.hero` pone blanco y el título lo hereda
`line-height`	Alto de línea	`1.6`, `1.15`	Ya viene en el reset
`text-decoration`	Subrayado	`none`, `underline`	El reset quita el de los enlaces

`text-align` vs `align-items` vs `justify-content`

Quiero centrar…	Uso	Dónde va
Las líneas de un párrafo	`text-align: center`	En el propio texto o en su contenedor
Una caja dentro de un padre flex	`align-items` / `justify-content`	En el **padre flex**
Una caja de ancho limitado	`margin: 0 auto`	En la propia caja (como `.contenedor`)

La herencia

Propiedades como `color`, `font-size`, `font-family` y `text-align` **se heredan**: si las pones en el padre, los hijos las reciben. Por eso pones `color` blanco en `.hero` y no lo repites en título y párrafo. `margin`, `padding` o `border` **no** se heredan.

5. La caja: espacio, bordes y tamaño

Todo elemento es una caja con cuatro capas, de dentro a fuera: **contenido → padding → border → margin**. El reset de `variables.css` pone `box-sizing: border-box`, así que el `width` incluye padding y borde.

Propiedad	Qué hace	Ejemplo en Marea
`padding`	Espacio **dentro** de la caja, entre el contenido y el borde	Botones, tarjetas, planes
`margin`	Espacio **fuera** de la caja, separa de los vecinos	`margin-bottom` del título de sección
`margin: 0 auto`	Centra una caja con ancho máximo	`.contenedor`
`border`	Borde: necesita **grosor + estilo + color**	`1px solid var(--color-borde)`
`border-radius`	Esquinas redondeadas	Tarjetas con `--radio-m`, botones con `--radio-pildora`
`box-shadow`	Sombra	Hover de las tarjetas con `--sombra-fuerte`
`overflow: hidden`	Recorta lo que se sale de la caja	Tarjeta: el color de la imagen respeta las esquinas
`width` / `max-width`	Ancho fijo / ancho máximo	`.contenedor` con `--ancho-contenedor`
`height` / `min-height`	Alto fijo / alto mínimo	Imagen de tarjeta, hero con `min-height`
`padding-block` / `padding-inline`	Padding solo arriba y abajo / izquierda y derecha	Secciones con `padding-block`

Atajos de `padding` y `margin`

Escribes	Significa
`padding: 1rem`	1rem en los 4 lados
`padding: 1rem 2rem`	1rem arriba y abajo, 2rem izquierda y derecha
`padding: 1rem 2rem 3rem 4rem`	Arriba, derecha, abajo, izquierda (en el sentido del reloj)

Errores típicos con `border`

- `border: 1px solid;` sin color usa el color del texto (`currentColor`).

- `border: var(--color-borde);` solo con color **no dibuja nada**: falta grosor y estilo.
- Para que un botón con borde mida igual que otro sin borde, dale `border: 2px solid transparent` a todos.

`height` vs `min-height`

Un `height` fijo recorta o desborda el contenido si hace falta más espacio. Un `min-height` es un suelo: la caja crece si el contenido lo necesita. Por eso el hero usa `min-height`.

Sticky footer

Con `body { display: flex; flex-direction: column; min-height: 100vh }` y `main { flex: 1 }`, el `main` se estira para ocupar todo el espacio libre y el pie queda siempre abajo, aunque `main` esté vacío. `100vh` es el 100% del alto de la ventana.

`position: sticky`

Solo la cabecera lo usa (`position: sticky; top: 0`): se mueve con la página hasta tocar el borde superior de la ventana y entonces se queda fija. `z-index` la pone por encima del resto del contenido al hacer scroll.

6. Fondos, colores y variables

Propiedad / función	Qué hace	Ejemplo en Marea
`background-color`	Color de fondo de la caja	`.hero` con `--color-tinta`
`background`	Atajo: color, degradado, imagen	Degradado de `.bloque__visual--playa`
`color`	Color del texto	Hero en blanco
`transition`	Anima un cambio (por ejemplo el hover)	Botones con `var(--transicion)`
`var(--nombre)`	Lee una variable CSS	En todo el archivo
`var(--a, valor)`	Variable con valor de reserva	Opcional
`clamp(min, ideal, max)`	Valor que crece con la ventana pero con límites	`--texto-3xl` del hero
`color-mix()`	Mezcla dos colores	Colores de hover derivados
`linear-gradient()`	Degradado entre colores	Fondos de playa y pinar

Propiedad / función	Qué hace	Ejemplo en Marea
`calc()`	Cuenta con unidades distintas	Opcional: `calc(100% - 2rem)`

Variables CSS

- Se **definen** con dos guiones: `--color-sol: #fcbf49;` dentro de `:root`.
- Se **usan** con `var(--color-sol)`.
- Si el nombre está mal escrito (por ejemplo `--text-xl` en vez de `--texto-xl`), el navegador **ignora esa línea sin avisar**. Es de los errores más difíciles de ver.
- Una línea rota en `:root` (por ejemplo `--color-destacado:` sin valor ni `;`) puede arrastrar la siguiente variable. Revisa el `;`.
- Las variables **no funcionan dentro de `@media (...)`**: ahí van los píxeles a mano.

Unidades

Unidad	Qué es
`px`	Píxeles fijos
`rem`	Múltiplo del tamaño de letra de la página (`1rem` = 16px por defecto): escala bien
`%`	Porcentaje del padre
`vh` / `vw`	1% del alto / ancho de la ventana
`fr`	Parte del espacio libre (solo en grid)

`transition` y hover

`transition: background-color var(--transicion)` hace que el cambio de color no sea instantáneo. Con `:hover` cambias el color y la transición lo suaviza. Sin movimiento (`transform`) cuando el enunciado lo prohíbe.

7. Selectores, pseudo-clases y cascada

Nomenclatura BEM

Las clases del HTML siguen el patrón `bloque__elemento--modificador`: **doble guion bajo** para una parte del bloque y **doble guion** para una variante.

Clase	Bloque	Elemento	Modificador
`.tarjeta__titulo`	tarjeta	titulo	—
`.boton--sol`	boton	—	sol
`.plan--destacado`	plan	—	destacado
`.bloque__visual--playa`	bloque	visual	playa

Si en el CSS escribes un solo guion bajo (`.cabecera_interior`), no coincide con el HTML y la regla no se aplica.

Selectores

Selector	Qué elige	Ejemplo
`.clase`	Los elementos con esa clase	`.tarjeta`
`.a .b`	Un `.b` que está dentro de un `.a` (descendiente)	`.pie .logo`
`.a > .b`	Un `.b` que es hijo **directo** de `.a`	`.menu__lista > li`
`.a, .b`	Los dos grupos a la vez	`.cartel, .escenarios, .entradas`
`.a + .b`	Un `.b` justo después de un `.a`	`.bloque + .bloque`
`.a.b`	Elementos con las dos clases	Sin espacio

Pseudo-clases y pseudo-elementos

Selector	Cuándo se aplica	Ejemplo en Marea
`:hover`	El ratón está encima	Botones y tarjetas
`:focus-visible`	Se navega con teclado (Tab)	Ya en `variables.css` ; se refuerza en fondos oscuros
`:focus-within`	Algo dentro tiene el foco	Tarjeta cuando se enfoca su botón
`:first-child` / `:last-child`	Primer / último hijo	Primer y último `<li>` del menú

Selector	Cuándo se aplica	Ejemplo en Marea
`:nth-child(4n + 1)`	Hijos 1, 5, 9… (ciclo de 4)	Colores de las imágenes de las tarjetas
`::before`	Inserta contenido antes del elemento	Viñeta "• " en la lista de planes

Cómo funciona `:nth-child(4n + 1)`

`n` empieza en 0 y sube: 4×0+1 = 1, 4×1+1 = 5, 4×2+1 = 9. Con cuatro reglas (`4n+1` , `4n+2` , `4n+3` , `4n+4`) los cuatro colores se repiten en ciclo y la séptima tarjeta recibe el color de la primera.

La cascada: quién gana

Si dos reglas afectan a la misma propiedad del mismo elemento:

1. Gana la más **específica** (un id pesa más que una clase, y una clase más que una etiqueta).
2. Si pesan igual, gana la que está **más abajo** en el archivo.

Tu caso real: un `.contenedor { display: flex; flex-direction: column }` escrito más abajo pisaba el `.cabecera__interior` de la cabecera, porque ese elemento también tiene la clase `contenedor` . Se arregla borrando la regla duplicada. En F12, la regla que pierde aparece **tachada**.

8. Errores típicos y cómo comprobarlos

Síntoma	Causa más probable	Arreglo
`gap` o `justify-content` no hacen nada	Están en un hijo sin hijos propios, no en el padre	Ponlos en el padre que tiene `display: flex/grid`
El grid no coloca las columnas	`display: grid` está en un contenedor que tiene un solo hijo	Aplica el grid al padre **directo** de los elementos (`.datos__lista` , no `.datos`)
No se centra el texto de un párrafo	Se usó `justify-content`	Usa `text-align: center`
La regla no se aplica	Nombre de clase mal escrito (`_` en vez de `__`)	Copia el nombre exacto del HTML

Síntoma	Causa más probable	Arreglo
El color no cambia	Variable que no existe (`--text-xl` , `--estado-...`)	Mira los nombres en `variables.css`
El borde no se ve	Solo se puso el color en `border`	`border: 2px solid var(--color-...)`
El fondo solo cubre el ancho del contenedor	El fondo está en el `.contenedor` , no en la sección	El fondo va en la sección; el ancho limitado, en el interior
Texto invisible en la cabecera	`color` blanco sobre fondo blanco	Cambia el color del texto
Los dos cuadros pasan a la izquierda	`order: -1` aplicado a todos los `.bloque__visual`	Solo en `.bloque--inverso .bloque__visual`
El botón de la tarjeta no queda abajo	`.tarjeta__cuerpo` no es flex en columna	`display: flex; flex-direction: column` + `margin-top: auto` en el botón

Comprobaciones del enunciado

1. Abre la página a **1440px y 1280px**: no debe haber scroll horizontal.
2. **Borra el texto** de una tarjeta: los botones deben seguir alineados abajo.
3. **Vacía el `<main>`** : el pie debe seguir al final de la ventana.
4. **Duplica una tarjeta** (la séptima): la última fila tiene una sola tarjeta y no se estira.
5. Navega con **Tab**: el foco debe verse siempre.
6. Pasa el ratón por botones y tarjetas: debe verse el hover.

Atajos de F12 para aprender

- Selecciona un elemento y mira el panel de estilos: la regla **tachada** es la que pierde en la cascada.
- Marca y desmarca propiedades (`display` , `gap` , `order`) para ver su efecto en directo.
- En el HTML, los elementos flex y grid muestran una insignia "flex" o "grid" que enseña la rejilla al pulsarla.